

Token entropy is computed incorrectly (counts of floats / indices).

In `main_batching.py` and `main.py`, `calculate_token_entropy` builds a histogram over *values* (floats or ids) rather than the **probability mass** over the top-k tokens. In `main_batching.py` it's even called with `outputs_probs_sort` (sorted token indices), so the function is taking the entropy of the *index set*, not of the model's probability vector. Replace with the Shannon entropy over softmax probabilities, e.g., $-\sum p_i \log p_i$ (optionally normalized by $\log k$). See this::

```
# Instead of entropy over counts of floats/ids ...
# token_entropy = calculate_token_entropy(outputs_probs_sort)

# Do this (assuming outputs_probs is a 1D tensor of probabilities for the last token):
topk = 10
vals, idxs = torch.topk(outputs_probs, k=topk, largest=True)
p = (vals / vals.sum()).cpu().numpy()
token_entropy = float(-(p * np.log2(p + 1e-12)).sum() / np.log2(len(p)))
```

Variable mismatch in `main.py` causes wrong prompts to be used.

Inside the loop I see the construct `entry = ...` but call the mechanism with `imagePrompt.prompt/imagePrompt.prefix` (undefined). This is a silent bug that can scramble results and visualizations. We can replace with `entry.prompt/entry.prefix`; likewise for the call to `save_vis`.

Inconsistent return signature of `get_attention_patches`.

Some versions return 2 items, others 4; callers expect `(demo_img, attn_24x24, token_probs, decoded_tokens)`. We can unify the API in both the single-image and batched versions (and keep top-k probabilities for entropy/calibration). Check the contradictory returns in `main.py` and the 3-tuple in batching.

Cluster entropy is stored in a field named `ave_strength`.

`cluster_entropy(...)` writes normalized entropy into `ClusterData.ave_strength`. This over-loads the field and makes downstream analysis easy to mix up. Create a separate `entropy` field and keep `ave_strength` for actual mean patch strength.

Clustering ignores the Z (attention) dimension

`find_clusters` uses 2D `(x,y)` only and simulates weights by duplicating points. Prefer a true weighted algorithm (DBSCAN supports `sample_weight`) or cluster in `(x, y, z)` with feature scaling; at minimum, pass `sample_weight` instead of duplication to avoid distortions. Current code: clusters on `(x,y)` only, duplicates in `duplicate_points`.

Head / hidden sizes are hard-coded

In batching: `HEAD_NUM=32`, `HEAD_DIM=128`. We need to derive these from the loaded model to keep experiments portable to 13B or variants.

Minor though : Safer HF integration

`setup.sh` overwrites files inside the installed `transformers` package. Consider subclassing or importing the local modules directly to avoid environment-wide side-effects during replication

Experiments Ideas

From the pdf, what I could gather is that the main claim is that **spatial attention clustering metrics have weak correlation with model confidence** (R^2 mostly < 0.3 ; Table 1 and discussion).

Robust clustering analyses (no new data needed)

- **3D or weighted DBSCAN + sensitivity sweep**
We can run DBSCAN on (x, y, z) after standardizing features (z = normalized attention) and sweep `eps` $\in [0.8, 2.0]$, `min_samples` $\in [5, 30]$. Track stability of: number of clusters, noise ratio, mean strength, and your normalized entropies; report R^2 /Spearman with token confidence. Current `find_clusters` is 2D only; add a 3D branch and a `sample_weight` path.
- **Replace duplication with `sample_weight`**
Use `DBSCAN(...).fit(coords, sample_weight=w)` where w is a monotonic transform of attention (e.g., $w = z^\gamma$, $\gamma \in \{0.5, 1, 2\}$). This removes duplication artifacts and will also help to understand how emphasis on strong patches (γ) affects correlations. Current duplication code: `duplicate_points(...)`.
- **Null model & permutation test**
Randomly permute the 24×24 attention map or rotate/flip it before clustering to **estimate a null correlation**. If the observed R^2 is not significantly higher than the null, the negative result is even stronger.
- **Alternative clusterers**
Add **HDBSCAN** (variable density) and **Gaussian Mixtures** on (x, y, z) . Try and report whether any non-DBSCAN method changes the story. The pipeline to 3D points is already modular (`transform_matrix_to_3d_points`).
- **Better entropy**
Compute entropy of the **normalized attention distribution** directly: $p_i = a_i / \sum a_i$ over the 576 patches. The current `calculate_entropy` counts identical float values, which makes entropy artificially high/flat. Fix this in both single and batched

Confidence variants (no new data)

- **Top-k margin & calibrated confidence**

Besides token entropy, we can add **margin** $p(\text{top1}) - p(\text{top2})$ and **logit margin**; and compute **ECE / Brier** on yes/no and short-answer subsets. We already have the softmax vector (`outputs["logits"][...]; get_prob`).

- **Answer-token vs. predicted-token deltas**

The head-score is the increase in log-prob for the **predicted** token after head manipulation. Also measure the delta for the **ground-truth** token in VQA 2.0 to see whether attention better explains *accuracy-aligned* confidence than *predicted-token* confidence. The head-perturbation logic (recomputing with `o_proj` and comparing to `origin_prob`) is already there.

- **Across generated tokens (not only the last)**

For multi-token answers, aggregate confidence across the whole answer span (mean log-prob or product). Currently I see that the code reads `outputs["logits"][0][-1]` (last token).
main

Image vs. language ablations

- **Language-only ablation**

Run with `pixel_values` zeroed or drop the image features right after the projector to quantify what fraction of correlation comes from language alone (atleast the overleaf I shared earlier stated ~85%). This is straightforward in the modified HF stack because we would already return rich caches/past values; just intercept before `multi_modal_projector` / merging

- **Visual dropout**

Randomly zero out a fraction of image patches before projection (e.g., 20%, 50%) to see whether spatial metrics become more/less predictive of confidence.

- **Head ablations**

We already compute head contributions by reconstructing with `o_proj` and measuring Δ log-prob. We can try and go one step further: **zero out** (or noise) the **top head** vs a **random head** and measure the change in confidence and in clustering metrics. If attention clusters were causal for confidence, removing the “important” head should move both confidence and spatial concentration in tandem. (we already select the top head via `all_head_increase_sort`.)

Prompt & prefix controls

- **Prefix sweep / no-prefix control**

Because the pipeline conditions on a category-specific prefix (e.g., “The color is ...”), we can run a *prefix-free* control and a *generic* prefix (“Answer briefly”) to check whether

linguistic scaffolding inflates confidence-only signals. The paper's Section 3.2 is what I am referring to

Minor thoughts/upgrades

- **Use `return_dict` and named fields**

The analysis indexes into tuples (e.g., `outputs[2]`) which is brittle across library versions; we can switch on `return_dict=True` and reference `outputs.past_key_values / logits` explicitly for clarity and reproducibility. (The custom `modeling_llama.py` stores extra per-layer tensors in the cache path already.)

- **Programmatically infer dims**

Replace `HEAD_NUM/HEAD_DIM` constants with values from the model config to keep 7B/13B runs consistent.

- **Patch size & resolution sweep**

we lock `patch_size=14` and crop to 336 to a 24×24 grid. Add runs at 224 and 448 to test whether coarse/fine patching changes spatial metrics' predictive power. (This is one flag change plus resize.)

- **Structured outputs**

Right now results print to stdout; dump a per-example JSON/CSV with: question type, token probs, margins, entropy, #clusters, noise ratio, per-cluster {size, mean z, entropy}, and $\Delta\log\text{-prob}$ for head ablations. That will let you reproduce the R^2 table in the paper at scale.

- I see the import `BitsAndBytesConfig` in batching but not used it; enabling 4-bit or 8-bit gets you higher batch sizes for full-val runs. Keep the logic in `main_batching.py`, which already collates batched prompts and images